



April 25 2026

Keeping a 20PF Slingshot Machine Performant

HPC System Test Workshop
CUG 2026

Jenett Tillotson

Senior HPC Systems Engineer: HPC Systems Group

Rory Kelly

HPC Consultant IV: Consulting Services Group

National Center for Atmospheric Research (NCAR)

- NSF funded institution for research geo-systems science with an emphasis on climate
 - Mainly studied the atmosphere, the ocean, the Sun, volcanoes, wildfires
 - Research other geo-systems science as well
 - Do the 100-year climate forecasts
 - Do twice daily weather forecasting for Antarctica
 - NOAA does the weather forecasting for the United States (not NSF NCAR)
- Eight labs to support this research
 - Atmospheric Chemistry Observations & Modeling (ACOM)
 - Climate & Global Dynamics (CGD)
 - Computational & Informational Systems Laboratory (CISL) <- my lab
 - Education, Engagement & Career Development (EDEC)
 - Earth Observing Laboratory (EOL)
 - High Altitude Observatory (HAO)
 - Mesoscale & Microscale Meteorology (MMM - pronounced "M-Cubed")
 - Research Applications Laboratory (RAL)

Derecho (NWSC-3)

2488 64-core Dual Socket Milan Nodes
82 GPU Nodes (328 A100 GPUs)



Casper

DAV/HTC/AI/ML Cluster



Bifrost

High-Bandwidth Low-Latency Ethernet Network Fabric (200 Gb/s/link)



Quasar

Tape Archive
(50 PB capacity)



Campaign Storage

(122 PB capacity)



Data Transfer Services

Globus, GridFTP



Stratus

Object Storage
(5 PB capacity)



Destor Lustre Scratch

(55 PB capacity)

- Sometimes you have to power down a perfectly working machine
- Things cool down and shrink causing hardware issues that affect network performance
 - cabling, optics, bus connections, etc.
- Heating and cooling cycles cause large amounts of stress on hardware
- Lessons learned: Never power down a supercomputer...
 - ...but that's not realistic

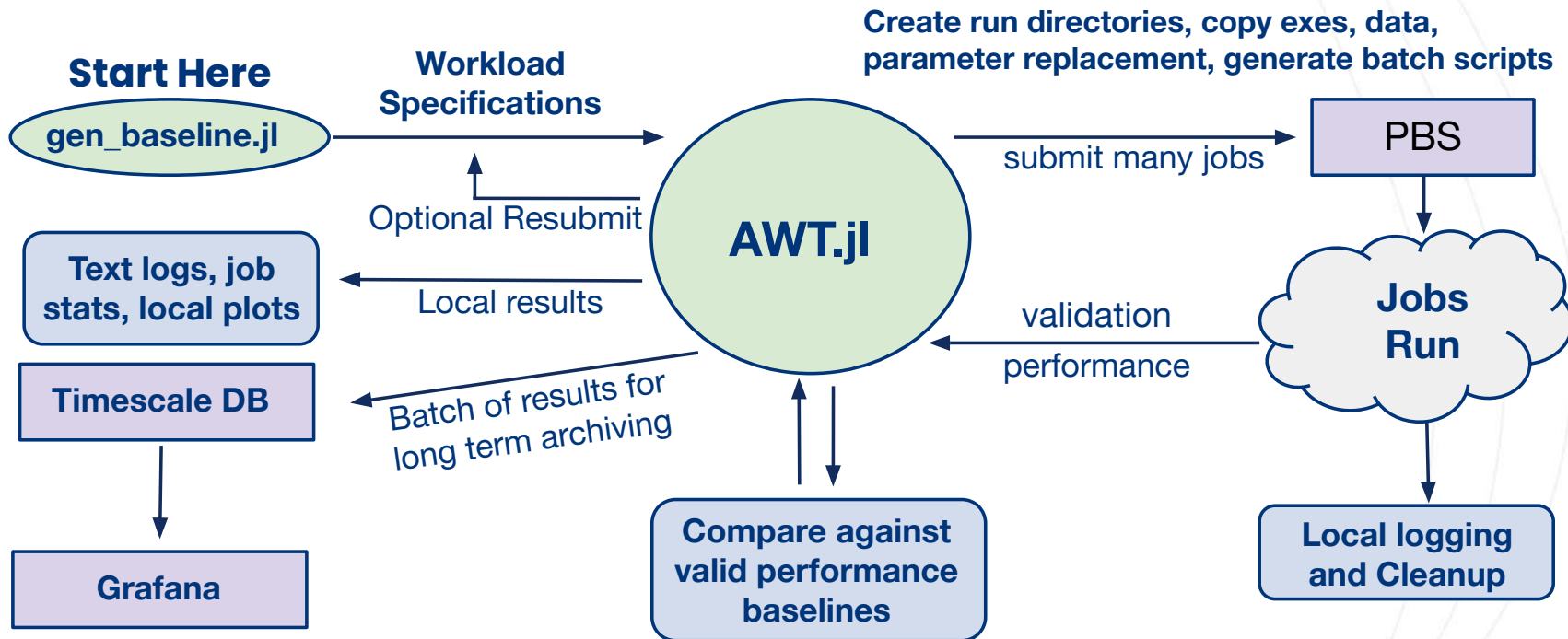
The team effort that is restarting a supercomputer

- Power everything on
- Hardware health checks (HPE checks)
- Configure nodes and bring into production
- Node-level software/hardware checks (node health checks)
- **Application level software/hardware checks**
- **Functionality of the user environment**

Application Workload Test (AWT)



Generate a synthetic workload using a mix of user applications and benchmarks.



- Full Applications

- HOMME
- LES
- MPAS-A
- POP
- WRF
- GOES
- FastEddy

- Synthetics and Kernels

- MG2
- CLUBB
- DART_WRF
- WACCM
- HPCG
- IOR
- Stream

- + Fairly realistic* user workload
- + High confidence that good results $\Rightarrow$ Healthy Machine
- + Required to track contractual performance over time
- - Can be slow to run (30+ minutes)
- - Requires some expertise or familiarity
- - Feedback not always immediately actionable (without further investigation)
- - Can be difficult to run in out of office scenarios

Summary: necessary for the case where changes are expected, but too heavy-weight when we don't expect changes

Desirables for a Streamlined Outage Tool



- Phone runnable
- Disambiguated tests of independent subsystems, per-node
- Concise and actionable output
- Can be run without expertise or experience
- Short end-to-end run-time
- Convoluted yet fun acronym:
Phone Runnable Application Yardstick For (measuring) Usability
of the System: **pray4us**

Overview of pray4us outage tool



- Single C++ MPI application running at full machine scale
 - Test results are reported for each node
- Current subsystem tests
 - CPU
 - Memory
 - Fabric
- Currently targets CPU nodes (more problematic than GPU)

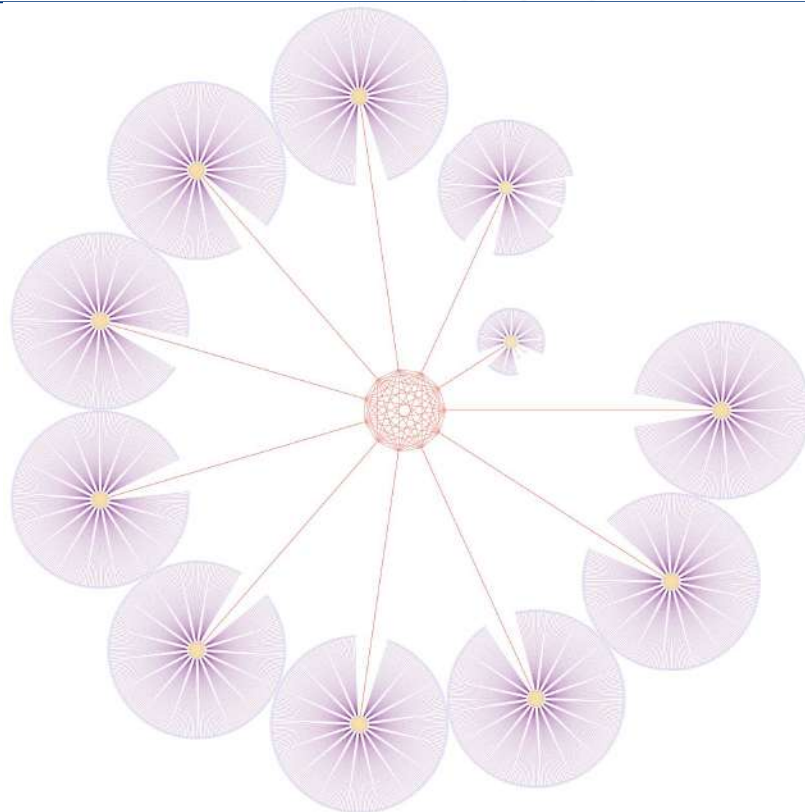
- Measures a timed kernel on each core of each node
 $x = \pi/2$, $\text{eps}=0.00005$
 $\text{while}(x > \text{eps}) x \rightarrow 0.5 * [\sin(x) + \cos(x-\pi/2)]$
stops after 1200705285 iterations (intel icx 2023) ~20 sec
- Repeats above test 3 times, look at timings allowing for some jitter
 - “small jitter” is 28 sec, “big jitter” is 60 sec
 - An individual test fails if there are
 - > 4 little jitter events
 - > 2 big jitter events
- Node is characterized as follows
 - Pass with 0,1 failures
 - Warning with 2 failures, rerun and see if it persists
 - Error with 3 failures, remove node and rerun

- Measures a Stream Triad ($C[i] = A[i] + \text{scalar} * B[i]$) on each core of each node, using 400 MiB arrays (~154 GiB / node).
- Sums bandwidth across all ranks on a node
- Node status characterized as
 - Good if bandwidth > 214 GiB/s
 - Error otherwise, remove node and rerun
- In practice, almost no variability in this test and I've never seen it fail
 - Only interesting thing was getting the C++ reimplementation to match the performance of the original C code
 - `std::aligned_alloc(64,...)`
 - `-Wstrict-aliasing` and `-qopt-assume-no-loop-carried-dep`

Derecho Slingshot Network Topology



- Eleven switch groups, all-to-all connected
- The “1 o’clock” group has room for expansion
- The “2 o’clock” group contains all the GPU nodes and nodes are quad injected
- The red ball in the middle represents the global connections between the local groups



Given slingshot, and the constraints of “short and simple”, what would be an interesting network test?

Arrange nodes in a 1d-torus, and do a 3-stage communication

- Communicate to the left (“close” on average, 1- or 2-hops)
- Communicate to the right (“close” on average, 1- or 2-hops)
- Communicate across the diameter (“far” always 3-hops)

Push enough traffic into the network to cause significant load.

- 128 ranks per node communicating the corresponding rank on a partner node.
- Each rank isends/irecvs 2 MiB, in 16 128 KiB chunks, and then Waitall.
- Measure aggregate node bandwidth. Barriers ensure the communication phases are synchronized.

Creating the Tori

```
// Run three offset patterns across the 1D torus.  
// Each iteration chooses a symmetric send/receive partner pair so  
// every node  
// measures the same logical distance in opposite directions with  
// wraparound.  
  
// For 1d torus topology, calculate nodes on left, right, and across  
// with iteration offsets  
// Left send: node_num-3, node_num-2, node_num-1  
// Right send: node_num+1, node_num+2, node_num+3  
// Across send: node_num+num_nodes/2-1, node_num+num_nodes/2,  
// node_num+num_nodes/2+1 (for i=0,1,2)  
// Receive partners must be symmetric: if I send distance d, I receive  
// from distance -d
```

- At full-machine scale, this communication pattern saturates at around 14,500 MB/sec/node on average.
- On each node, a communication phase has passes if measured bandwidth is > 14,000 MB/sec/node, and fails otherwise.
- Failures are tracked separately for left/right/across phases. Failures sending left/right are very rare, failures sending across happen occasionally, even when the fabric is in a “healthy” state.
- Look at failure modes on each node, attempt to give actionable feedback.

Examples of failure modes, and actions

Per node:

1 failure across → Pass, this happens sometimes

1 failure right or left → Warn, could be a fluke, or problem with a partner node

2 across → Warn, could be a fluke

1 across and 1 left/right → Warn, could be a fluke

2 left or 2 right → Error, maybe a problem with a partner node

1 left and 1 right → Error, maybe a problem with this node

3 or more failures → Error, maybe a problem with this node

Actions: on warning, recommend rerunning to see if problem persists on error,
recommend taking node offline and rerunning

Globally:

<= 20 failures across → Pass, pretty normal

> 20 <= 40 failures across → Warn, remedy node-level issues and rerun

> 40 failures across → Error, something is off, brew coffee, fall back to AWT.

pray4us Example Output



===== Fabric Test =====

- These nodes triggered a warning condition
rerun and set offline if warnings persist
 - dec0980
 - dec2222

===== Memory Test =====

- No warnings or errors found

===== Compute Test =====

- These nodes triggered an error condition
set offline and rerun the test
 - dec1957

===== Test Summary =====

Fabric test status: PASSED

Memory test status: PASSED

Compute test status: FAILED

pray4us Example Output



===== Fabric Test =====

- No warnings or errors found

===== Memory Test =====

- No warnings or errors found

===== Compute Test =====

- No warnings or errors found

===== Test Summary =====

Fabric test status: PASSED

Memory test status: PASSED

Compute test status: PASSED

- Get more experience
- Add equivalent tests for the GPU nodes
 - Expect to need different tunings, but straightforward
- Implement more sophisticated statistical criteria for CPU/Fabric tests
- Contemplate an I/O test

QUESTIONS?